

Introduction

This report documents five controlled experiments designed to isolate and evaluate specific architectural decisions in the Intellisys pipeline. Each experiment compares two configurations of the same system on the same set of objectives. No benchmark numbers are invented in this report: where an experiment has not yet been executed against a live OpenRouter/Tavily configuration, the Expected Outcome section states a hypothesis grounded in the implementation, and the Discussion section states plainly that results are pending live execution.

Experiment 1: Single Agent vs Multi-Agent

Objective

Determine whether decomposing research into five specialized agents (Supervisor, Research, Analyst, Critic, Writer) produces measurably better reports than a single agent performing planning, searching, analysis, and writing in one pass.

Methodology

Two configurations are compared on the same set of objectives from the Evaluation Dataset: (A) the existing multi-agent LangGraph pipeline, and (B) a single-agent baseline that receives the objective and is asked to directly produce a final Markdown report in one LLM call, with tool access to Tavily search but no separate planning, analysis, or critique stages.

Procedure

- Select 8-10 representative objectives spanning technical, comparative, and business-decision categories from the Evaluation Dataset.
- Run each objective through configuration (A), the existing pipeline, recording the final report.
- Implement a minimal single-agent baseline (B) using the same LLMClient and web_search tool, with one combined prompt.
- Run the same objectives through configuration (B), recording the final report.
- Score both sets of reports on: citation accuracy (do referenced URLs correspond to real retrieved evidence), structural appropriateness (does the report structure match the topic type), and coverage of the original objective's sub-questions.

Expected Outcome

The multi-agent pipeline (A) is expected to produce reports with more reliable citations, since evidence retrieval and citation are structurally separated from generation and the References section is deterministically built from real database rows — a property the single-agent baseline (B) cannot replicate without equivalent scaffolding. (A) is also expected to show better topic-appropriate structuring, since the Analyst and Writer stages are each given a narrower, more specific task than a single combined prompt.

Discussion

This experiment has not yet been executed; building the single-agent baseline (B) requires new code not present in the current implementation. The expected outcome above is a hypothesis derived from the architectural differences already documented (specifically, that the References section in the multi-agent system cannot contain a hallucinated source by construction, while a single-agent baseline has no equivalent safeguard).

Conclusion

Pending execution. The architectural argument for multi-agent decomposition is strongest for citation reliability, since that property is enforced by code structure (a separate, non-LLM-generated References section) rather than by prompting alone — this is a testable, falsifiable claim once the baseline exists.

Experiment 2: Critic Enabled vs Critic Disabled

Objective

Measure the effect of the Critic Agent's review-and-revision loop on final report quality, isolating the specific contribution of that one stage.

Methodology

Two configurations: (A) the standard pipeline with the Critic active and able to trigger up to `max_revisions=2` Analyst revisions, and (B) an identical pipeline with the Critic node bypassed entirely — the Analyst's first output is passed straight to the Writer with no review.

Procedure

- Select objectives likely to produce borderline-quality first-pass analyses (e.g. topics with thin evidence, such as EVAL-011).
- Run each objective through configuration (A), recording whether the Critic ever returns `REVISION_REQUIRED` and how the revised analysis differs from the original.
- Run the same objectives through configuration (B) (Critic bypassed), using the same underlying evidence.
- Compare the number of `unsupported_claim_warnings` present in the final analysis used by the Writer in each configuration, and manually review whether (B)'s reports contain claims that (A)'s revision process caught and corrected.

Expected Outcome

Configuration (A) is expected to produce analyses with fewer unaddressed `unsupported_claim_warnings` by the time they reach the Writer, particularly on thin-evidence topics, since the Critic's explicit purpose is to catch exactly this class of problem before it reaches the report.

Discussion

Not yet executed. This is the most directly testable of the five experiments using the existing codebase, since disabling the Critic only requires bypassing one conditional edge in the graph (route directly from analyst to writer) rather than building new components.

Conclusion

Pending execution, but architecturally straightforward to run. This experiment is the recommended first one to execute, given the low implementation cost of configuration (B).

Experiment 3: Sequential vs Parallel Research

Objective

Quantify the wall-clock time savings from executing research questions concurrently (via `asyncio.gather()`) versus sequentially, and confirm parallelism does not change what evidence is retrieved.

Methodology

Two configurations: (A) the existing `ResearchAgent.research_all()`, which fires all Tavily searches concurrently, and (B) a sequential variant that awaits each search one at a time in a simple loop.

Procedure

- Select a small number of objectives with plans containing the maximum 6 research questions, to maximize the visible effect of parallelism.
- Run each objective through configuration (A) and record the wall-clock time for the research stage specifically (from `research_node` entry to exit).
- Run the same objectives through a sequential variant (B) of the same code, changing only `asyncio.gather()` to a sequential `await` loop.
- Compare stage duration between (A) and (B), and confirm the sets of stored evidence rows are equivalent in content (allowing for natural result variability from re-querying Tavily).

Expected Outcome

Configuration (A) is expected to complete the research stage roughly proportional to the slowest single Tavily call, rather than the sum of all calls, since `asyncio.gather()` runs them concurrently — for 6 questions this should be a substantial multiple faster than (B) if individual call latency is roughly uniform.

Discussion

Not yet executed with timing instrumentation. The theoretical expectation follows directly from `asyncio.gather()` semantics and does not depend on LLM behavior, making this the experiment with the most predictable outcome of the five — the main open question is how much per-call latency variance exists in practice.

Conclusion

Pending timed execution, but expected to strongly favor (A) on wall-clock time with no expected difference in evidence quality or completeness, since parallelism here only affects timing, not what is retrieved.

Experiment 4: Full Context vs Role-Specific Context

Objective

Determine whether giving each agent only the narrow, role-specific context it needs (the current design) produces better outcomes than giving every agent the full conversation history and all intermediate artifacts.

Methodology

Two configurations: (A) the existing design, where WorkflowState passes only explicitly typed fields to each node (e.g. the Analyst receives evidence but not the Supervisor's internal reasoning), and (B) a variant where every agent's prompt is prefixed with the full accumulated state (objective, plan, all evidence, all prior agent outputs) regardless of relevance to that agent's task.

Procedure

- Implement configuration (B) by modifying each agent's prompt-construction step to include the full WorkflowState rather than its currently-scoped subset.
- Run a shared set of objectives through both (A) and (B).
- Compare token usage per call (a direct cost proxy), and manually review whether (B)'s outputs show any quality improvement or, conversely, any signs of the agent being distracted by irrelevant prior content (e.g. the Writer echoing the Supervisor's internal rationale instead of writing the report).

Expected Outcome

Configuration (A) is expected to use meaningfully fewer tokens per call than (B), since role-specific context is smaller by construction, while producing comparable or better output quality — the current design's typed WorkflowState fields already reflect a deliberate choice to keep each node's input minimal, per the Architecture Documentation.

Discussion

Not yet executed. Building configuration (B) requires temporarily removing the existing context-scoping logic, which is a deliberate architectural boundary in the current code (`app/graph/state.py`) rather than an oversight — this experiment is partly a validation that the existing narrow-context design was the right call.

Conclusion

Pending execution. The prediction favors (A) on both cost and focus grounds, consistent with the stated design rationale in the Architecture Documentation, but this has not yet been empirically confirmed against (B).

Experiment 5: Revision Limit Comparison

Objective

Evaluate the effect of different values of `max_revisions` (the Critic-to-Analyst loop cap) on final report quality and total workflow cost/time.

Methodology

Three configurations tested on the same objectives: `max_revisions=0` (Critic can never trigger a revision — first analysis always proceeds), `max_revisions=2` (current default), and `max_revisions=4` (a higher cap allowing more revision cycles).

Procedure

- Select objectives from Experiment 2's set that are likely to trigger at least one revision under the default cap.
- Run each objective three times, once per `max_revisions` value, using the existing `Settings.MAX_REVISIONS` configuration.
- Record: total number of Analyst+Critic round trips actually used, total wall-clock time, total LLM call count, and a manual quality comparison of the final report across the three runs.
- Specifically check whether reports from `max_revisions=4` show meaningfully fewer `unsupported_claim_warnings` than `max_revisions=2`, or whether quality plateaus before the cap is reached.

Expected Outcome

`max_revisions=0` is expected to be fastest and cheapest but most likely to carry forward unaddressed unsupported claims. `max_revisions=2` (current default) is expected to strike a practical balance. `max_revisions=4` is expected to cost more in time and LLM calls with diminishing quality returns once the Critic's concrete issues have already been addressed in the first revision or two — most quality gains from a Critic loop are expected to occur in the first revision cycle, with later cycles addressing progressively smaller issues.

Discussion

Not yet executed. `MAX_REVISIONS` is already externalized as a `Settings` value, making this the cheapest of the five experiments to run operationally (a configuration change, not a code change) — it is recommended as a quick follow-up once Experiment 2 has established a baseline revision rate.

Conclusion

Pending execution. The current default of 2 is a reasonable starting assumption consistent with the 'diminishing returns' hypothesis above, but the actual optimal value should be set from measured quality-per-additional-revision data rather than left as an unvalidated default.